Recommending TV Programmes at sweet.tv: Repeat Viewing, Graph and Content Methods, and a Learned Re-ranker

Student: Keshav Gupta Team: Individual
Email: kgupta@edu.hse.ru Date: 2026-10-30
MidTerm: TV-programme recommendation on sweet.tv (Kaggle competition)
Code and final submission file: https://github.com/keshavgupta8055/RecSys_Midterm

Abstract

People who watch TV mostly come back to the same programmes, so predicting the five programmes each sweet.tv user will watch in later weeks is mainly a question of how to use their own history. I compared a popularity list, a personal-frequency list, Personalized PageRank, content-based filtering and a schedule-exposure ranker, and combined them in a LightGBM LambdaRank re-ranker trained on an earlier time window. On a temporal validation split the re-ranker reached MAP@5 0.3482, and it was also the best of my three Kaggle submissions. The main takeaway is that a user's own history and the upcoming schedule explain most of the accuracy, while the learned part adds a small gain and recommends few new programmes.

Index Terms — repeat consumption, Personalized PageRank, content-based filtering, learning to rank, LightGBM

1. Introduction

Problem statement. The sweet.tv Kaggle competition gives a log of viewing sessions and the broadcast schedules of about 400 channels. For each of 1,260 test users I must predict a ranked list of five programmes (tv_show_id) that the user will watch most in weeks 31-42. The score is MAP@5. A programme counts as watched in an airing if the user saw at least 80% of that airing, and the truth is the five programmes with the most such airings.

Motivation. Only about 11% of schedule rows carry a programme id, the test weeks are later than all training weeks, and users watch the same programmes again and again. So it is not obvious which signals help: the user's own history, co-viewing with other users, programme metadata, or the schedule itself.

Concrete example. Take one light user (id 385172162373391221). In weeks 11-20 this user had 17 programmes with at least one good airing; the top ones were a series (21 airings) and the news show ТСН (9 airings). In weeks 21-30 the real top-5 was three news shows (Факты, ТСН, Викна-Новости) and two films the user had not watched before (Мумия возвращается, Библиотекарь 2). A personal-frequency list puts the series first, so it misses; the news shows are hits; the two films cannot be guessed from this user's history.

Contributions.

- A documented join and validation protocol, and a comparison of baselines, two methods (Personalized PageRank, content-based filtering) and a LightGBM re-ranker on one temporal hold-out.
- A final strategy (the re-ranker) with its Kaggle score, checks against leakage and bugs, and an honest account of its limits.

2. Related Work

Prior work consulted: [1] Personalized PageRank (random walk with a jump distribution); [2] content-based recommender systems; [3] the LambdaRank family and [4] LightGBM, used for the re-ranker; [5] reciprocal rank fusion, which inspired my non-learned fusion baseline (mine uses tuned weights, so it is not a reproduction of [5]); [6] SHAP, for the feature-effect plots (I use SHAP-style contributions from LightGBM, not the original method); [7] the competition page.

Alternatives considered (not implemented; I make no claim about how well they would work here):

- Matrix factorization for implicit feedback: not implemented because of time; I chose two other methods from the exam list.
- Association rules: not implemented because of time; same reason.
- Sequence models: out of scope for this exam.

3. Method

Approach. I turn the view log into user-programme interactions with a time-overlap join, build a per-user history, and rank candidate programmes with several simple scorers. A LightGBM LambdaRank model then re-ranks the candidates using features from all scorers, and the top 5 become the list. Every method uses the same pool, the same rule for repeats (allowed) and the same fallback.

Pipeline.

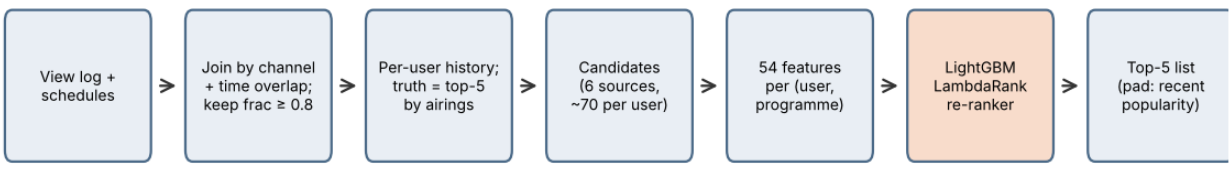


Figure 1. Pipeline from raw files to a top-5 list.

Tools and libraries.

- Python 3.13.1, pandas 2.3.1, numpy 2.3.2, pyarrow 21.0.0, scipy 1.16.1, scikit-learn 1.6.1

• LightGBM 4.7.0 (re-ranker), matplotlib 3.10.5 (figures), sympy 1.14.0 (exact toy checks), psutil 7.0.0 (memory)

Key design decisions.

• **Matching rule.** Views of one user on one channel are merged when they overlap; the view end is start + duration (117 views had stop - start off by 3600 s at the daylight-saving change); frac = watched / airing length, and frac ≥ 0.8 counts. tv_show_id = 0 rows are unlabelled airtime and are never recommended. Users with no usable history get the recent-popularity list, which is also the padding for every method.

• **Exposure feature.** The number of airings of a programme in the target window on the user's channels. Only airing counts are used; the target-period schedule is never joined to anyone's viewing.

• **No fitting on validation.** The re-ranker is trained on an inner task (history weeks 11-15, target weeks 16-20) and tuned by user-grouped cross-validation there, so validation labels are never fitted.

• **Repeats allowed,** for every method, because repeat viewing is most of the signal.

Configuration.

Seed 42 everywhere; bootstrap 1,000 user resamples. Method A (PPR): damping 0.85, edge-weight parameter 0.5, personalised on the history. Method B (CBF): TF-IDF on metadata with title words, profile from airings with frac ≥ 0.8 (counted), category weight 0.5; the blend with a popularity prior uses 0.35. Exposure baseline M0: gamma 0.25. Re-ranker M3: LightGBM lambdarank, 31 leaves, learning rate 0.1, min child samples 20, 59 trees (early stopping 50 on a user-grouped 20% hold-out, then refit); 54 features in 8 groups (history strength 10, recency 5, exposure 12, PPR 2, CBF 5, popularity 5, user context 5, programme attributes 10). Hardware: Windows 11, 6 cores / 8 threads, 7.69 GB RAM; peak memory below 2 GB; full pipeline 67-84 minutes.

4. Experiments

4.1 Setup.

Data: the view log (5.82 M rows) and three schedule files. Validation: history weeks 11-20, target weeks 21-30, 4,030 users (ALL) and a subset of 1,150 (SUB); pool 6,376 programmes; time order is kept. MAP@5 is the standard Kaggle AP@K (assumed, not checked); 95% intervals come from a user bootstrap.

4.2 Results.

method	role	ALL [95% CI]	SUB	M3 minus method (ALL)
pop_recent	baseline	0.2398 [0.2328, 0.2466]	0.2656	+0.1084
PF_softfrac	baseline	0.3237 [0.3168, 0.3311]	0.3600	+0.0245 [0.0213, 0.0274]
PPR	method A	0.3256 [0.3188, 0.3331]	0.3640	+0.0225
content-based, pure	method B	0.1815 [0.1761, 0.1873]	0.1992	+0.1667
content-based + popularity	method B	0.2783 [0.2717, 0.2849]	0.3111	+0.0699
PF + exposure (M0)	baseline	0.3406 [0.3336, 0.3482]	0.3803	+0.0075 [0.0050, 0.0099]
score fusion (M1)	hybrid	0.3416 [0.3346, 0.3490]	0.3803	+0.0066 (point diff.)
logistic regression (M2)	hybrid	0.3421 [0.3349, 0.3497]	0.3780	+0.0061 (point diff.)
LambdaRank (M3)	**final hybrid**	**0.3482 [0.3412, 0.3556]**	**0.3880**	**-**
LightGBM binary (M4)	hybrid	0.3479 [0.3406, 0.3552]	0.3881	+0.0003 (point diff.)

Table 1. Validation MAP@5. Against M0 the paired differences of M1, M2, M4 are +0.0010, +0.0015, +0.0073 (only M3 and M4 have intervals above 0). A history-only ranker can reach at most 0.6532 (ALL).

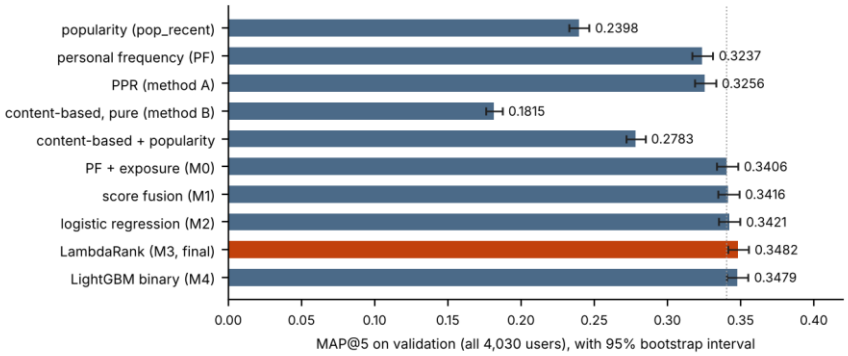


Figure 2. MAP@5 with 95% intervals; the dotted line is M0.

file	method	validation (ALL)	Kaggle private	Kaggle public
sub02_personal_baseline	PF_softfrac	0.3237	0.39369	0.41112
sub05_pf_exposure	M0	0.3406	0.41766	0.42602
sub06_hybrid (final)	**M3 LambdaRank**	**0.3482**	**0.43656**	**0.44181**

Table 2. Scores read from the Kaggle Submissions page (Figure 3).

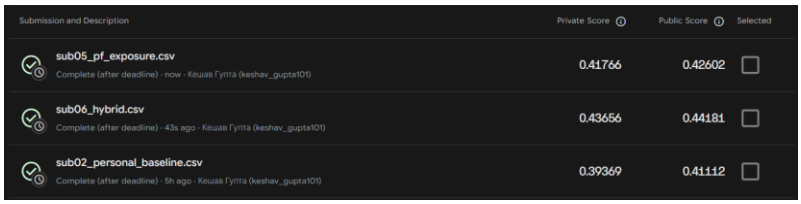


Figure 3. Kaggle Submissions page (account keshav_gupta101): private score of the final file sub06_hybrid.csv is 0.43656.

4.3 Comparison with the baseline.

The re-ranker M3 beats the popularity baseline by +0.1084 MAP@5 and the personal-frequency baseline by +0.0245. Against the strongest simple baseline M0 the gain is +0.0075 with a paired interval of [0.0050, 0.0099]. On Kaggle the order is the same as on validation: sub06 > sub05 > sub02. Absolute Kaggle scores are higher than validation, because the test history is longer and the users differ; I do not compare them across the two.

4.4 Verification.

(a) Toy examples for PPR and CBF were solved by hand and matched the code; AP@5 was worked out by hand for one user and matched. (b) A slow pure-Python version of the join (19,767 user-programme pairs), of PPR, of CBF and of 31 history and exposure features agreed with the fast code (largest feature difference 3.6e-15). (c) Leakage tests: the re-ranker trained on shuffled labels scores 0.0535, random re-ranking inside the candidates 0.0223, and the real model against truth shuffled across users 0.1617, all far below the real 0.3482. (d) Moving the history end one week back changes 12.5% of (user, programme) history pairs. (e) Two full runs from scratch produced identical result files (hashes).

4.5 Two example users and business analysis.

I chose a light user (17 qualifying programmes in training) and a heavy user (id 5672174985970291620; at least 10 programmes with 4 or more qualifying airings) so that the methods are compared on very different histories. Tables 3 and 4 show every method's top-5 with ids; green and bold = in the user's true top-5. All lists have five items.

#	popularity	PF	PPR	CBF pure	CBF+pop	M0	final M3
1	**2400480 ТСН. (Info)**	10003159 Ничто не слу… (Ser.)	**2400480 ТСН. (Info)**	10003159 Ничто не слу… (Ser.)	**2400480 ТСН. (Info)**	**2400480 ТСН. (Info)**	**2400480 ТСН. (Info)**
2	**240081 Факты. (Info)**	**2400480 ТСН. (Info)**	10003159 Ничто не слу… (Ser.)	5900347 'Деньги'. (Info)	**240081 Факты. (Info)**	**240081 Факты. (Info)**	**240081 Факты. (Info)**
3	**2400467 'Викна-Новости'. (Info)**	**240081 Факты. (Info)**	**240081 Факты. (Info)**	10002697 Позднее раск… (Ser.)	10003159 Ничто не слу… (Ser.)	10003159 Ничто не слу… (Ser.)	2400508 Сегодня. (Info)
4	2400508 Сегодня. (Info)	2400508 Сегодня. (Info)	2400508 Сегодня. (Info)	90073285 Х/ф 'Моя новая жизнь', 1-… (Film)	2400508 Сегодня. (Info)	2400508 Сегодня. (Info)	**2400467 'Викна-Новости'. (Info)**
5	20088 'Завтрак с 1+1'. Информац… (Ent.)	**2400467 'Викна-Новости'. (Info)**	**2400467 'Викна-Новости'. (Info)**	90077794 Х/ф 'Поделись счастьем св… (Film)	**2400467 'Викна-Новости'. (Info)**	**2400467 'Викна-Новости'. (Info)**	10003159 Ничто не слу… (Ser.)

Table 3. Light user. Truth: Факты, ТСН, Викна-Новости, Мумия возвращается, Библиотекарь 2. Hits: popularity 3, PF 3, PPR 3, CBF pure 0, CBF+pop 3, M0 3, M3 3.

#	popularity	PF	PPR	CBF pure	CBF+pop	M0	final M3
1	**2400480 ТСН. (Info)**	**2400480 ТСН. (Info)**	**2400480 ТСН. (Info)**	5900319 Абзац! (Info)	**2400480 ТСН. (Info)**	**2400480 ТСН. (Info)**	**2400480 ТСН. (Info)**
2	240081 Факты. (Info)	**2400467 'Викна-Новости'. (Info)**	**2400467 'Викна-Новости'. (Info)**	5900347 'Деньги'. (Info)	**2400508 Сегодня. (Info)**	**2400467 'Викна-Новости'. (Info)**	**2400467 'Викна-Новости'. (Info)**
3	**2400467 'Викна-Новости'. (Info)**	6500479 'Мир наизнанку'. (Edu.)	6500479 'Мир наизнанку'. (Edu.)	500368 Ток-шоу '60 минут' с Ольг… (Ent.)	**2400467 'Викна-Новости'. (Info)**	**2400508 Сегодня. (Info)**	**2400508 Сегодня. (Info)**
4	**2400508 Сегодня. (Info)**	**2400508 Сегодня. (Info)**	**2400508 Сегодня. (Info)**	500346 'Право на власть'. (Ent.)	240081 Факты. (Info)	6500479 'Мир наизнанку'. (Edu.)	6500479 'Мир наизнанку'. (Edu.)
5	20088 'Завтрак с 1+1'. Информац… (Ent.)	500331 'Один за всех'. (Ent.)	240081 Факты. (Info)	**2400508 Сегодня. (Info)**	5900347 'Деньги'. (Info)	12002805 М/с 'Моланг'. (Kids)	240081 Факты. (Info)

Table 4. Heavy user. Truth: ТСН, Секретные материалы, Викна-Новости, Сегодня, Т/с 'Короткое слово нет'. Hits: M3 3 (AP 0.60), same as M0 and CBF+pop; PF and PPR 3 (AP 0.55); popularity 3 (0.48); CBF pure 1.

Which strategy would I select, and why? The LambdaRank re-ranker, with M0 as the simple fallback. It has the best validation score and the best Kaggle score, and its cost at run time is small (final fit 9 s, inference under 1 s; M0 needs only counting tables, 18.6 s on validation). Its price is tuning time (about 36 minutes for the whole step).

Familiar or new? 96.3% of the final lists are programmes the user already watched in training (M0: 99.1%); precision is 0.417 on familiar and 0.133 on new programmes, and only 1.2% of hits are new. In both example users the truth contained programmes no method found (the two films of the light user, the series of the heavy user). Pure content-based filtering recommends unseen series and films, but gets 0 hits for the light user and 0.1815 MAP@5 overall. An unseen programme is not automatically a useful one, and not being in the log does not prove the user never saw it on another service.

How the methods differ. Popularity already does well for news: for the light user the popularity list gets 3 hits and AP 0.60, higher than the re-ranker's 0.55, but over all users it is far lower (0.2398). PPR is almost the same as personal frequency, because the co-view graph is 82% dense and 99% of its recommendations are already in the user's history. Content-based filtering helps only when it is blended with popularity or used as a feature (ablation +0.0054 when added to history alone). Schedule exposure helps most: adding it to a history-only model gives +0.0177, and removing it from the re-ranker costs 0.0081 (Figure 4). I did not implement association rules or matrix factorization, so I say nothing about what they would capture.

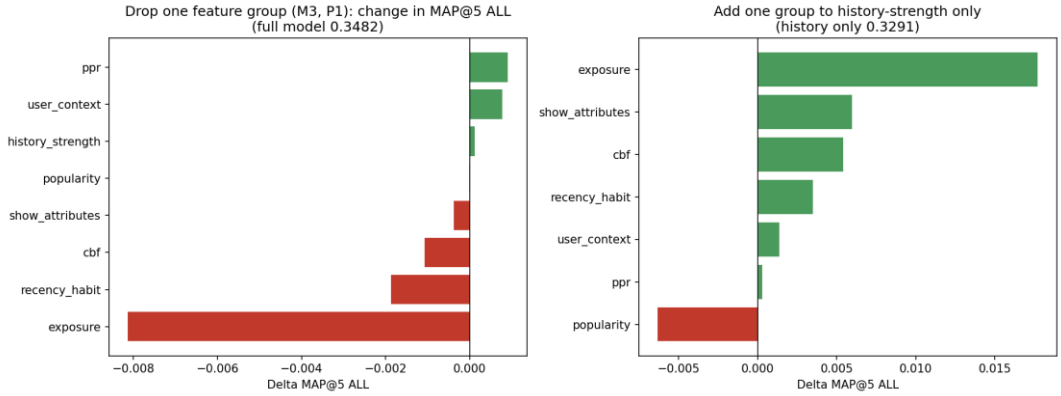


Figure 4. Left: change in MAP@5 when one feature group is removed from the final model. Right: change when one group is added to a history-only model.

What the lists mean for users. On validation the final model recommends only 163 different programmes (2.56% of the pool; M0 421). Info programmes are 52.8% of its recommendations against 28.1% of the truth, and Films 0.24% against 19.5%. It keeps viewers on what they know, but it is not a discovery engine. These are measured on historical logs only; I make no claim about user enjoyment or revenue.

5. Discussion

Failure case.

During the content-based step the run stopped on my own cross-check: AssertionError at src/run_step5.py line 312, assert (nv.max_abs_diff < 1e-10).all() and nv.top5_identical.all(). The vectorised top-5 and the slow pure-Python top-5 disagreed for user 1697310425691724323.

Root cause.

Programmes 12002450 and 12002526 have identical item vectors, but their cosine scores differed by about 1e-16 (0.52497512637251020 against ...31) because of floating-point summation order. The fast code ordered them by this noise and the slow code the other way, instead of by the intended tie-break (recent popularity).

Fix and verification.

Detection: the naive cross-check on 10 users. Change: scores are rounded to 12 decimals before ranking, in both implementations. Confirmation: the step re-ran, the assertion passed, vectorised and naive top-5 lists were identical, cbf_pure ALL moved from 0.1814 to 0.1815, and the share of ties at the 5th/6th place rose from 9.28% to 11.36%, which shows how often noise had been deciding ties. Another bug of the same family (user ids silently turned into floats, which first made a brute-force join check pass for no reason) was caught the same way; all bugs are in outputs/LOG.md.

A model-level failure is a user with no history (id 18133386382515269463): the re-ranker put SpongeBob first and dropped Викна-Новости, which M0 had at rank 3 and which was in the truth. The SHAP-style contributions show the global airing rate (+1.71) and the popularity score (+0.56) dominating. My explanation is that, with no history, these are the only strong signals left; I did not test this explanation with an experiment.

What worked.

- Target-period exposure (largest single gain) and the inner task, which kept validation labels out of training.
- Naive cross-checks, which caught the tie-break bug and the float-id bug.
- Reporting intervals: they showed that M1, M2 and the PPR gain are not clearly better than M0.

What surprised me or did not work.

- PPR is almost the same as personal frequency (82% dense graph), and pure content-based filtering is below popularity.
- Popularity features hurt a history-only model (-0.0063 when added).
- The re-ranker is better on average but 55.6% of users get exactly the same score as with M0, and it recommends few programmes.

Next improvement.

For users without history, cap or switch off the global exposure features and add a source for films and new programmes; test it with the same inner/validation protocol and compare ALL MAP@5, the share of Films in the lists and the number of distinct programmes.

Limitations.

- Only about 11% of schedule rows have a programme id, and 41.2% of watch time is on unlabelled airings, which no method can see.
- The Kaggle tie rule and the exact MAP@5 definition are assumed (standard AP@K); my tie rule is count, watched seconds, then id.
- Weeks 25-27 of the validation target have about half the usual viewing, and one day of the log is missing.
- 10.6% of validation truth pairs are programmes nobody watched in training (recall 0 for all history methods).
- Settings of PPR, CBF and fusion were chosen on validation, so those numbers are optimistic (the re-ranker was tuned on the inner task). The Kaggle halves have about 630 users each, and I did not measure what difference that size can show.

Conclusion.
Repeat viewing is the core of this task: a user's own history beats popularity by a wide margin (0.3237 against 0.2398), and the upcoming schedule adds a little more (0.3406). The LightGBM re-ranker that mixes history, schedule, graph and content features is the best model I built (0.3482 on validation, 0.43656 on Kaggle private), but its gain over M0 is small and it narrows what users see. I would use it as the main strategy, keep M0 as a fallback, and work next on users without history and on films.

6. AI Usage Disclosure

AI tools used.
- I used Claude Code (Opus 5.5) inside VS Code. I wrote step-by-step instructions (seven steps, each with rules, required checks and outputs); Claude Code wrote and ran the code, made the tables and plots and wrote the log files. I checked the steps myself: I worked out the toy examples for PPR and CBF and the AP@5 of one user by hand, and my numbers matched the code. I uploaded the three submissions to Kaggle by hand. I did not independently verify the Kaggle metric definition, tie handling, or the internals of the libraries. The original session export is submitted as the second file.

How AI was used.
- Code generation: the whole pipeline (join, labels, baselines, PPR, CBF, features, re-ranker, submission tools). Debugging: the bugs listed in outputs/LOG.md. Visualization: 40 figures. Docs: LOG.md, PROTOCOL.md, README. Search: reference lookup and checking (manually as well).

What I personally verified.
- Worked the PPR and CBF toy examples and the AP@5 of one user by hand; the results matched the code.
- Checked the outputs of steps 1-7 myself, including the hand calculations above.
- Opened all seven references and checked authors, titles, venues and links.
- Read the Kaggle scores from the Submissions page myself (Figure 3).

What I trusted without verification.
- The Kaggle MAP@5 definition and tie handling; the internals of LightGBM, pandas, scikit-learn and scipy.
- The features outside the 31 history/exposure features that were recomputed by a naive implementation (their checks are the leakage tests only).
- That the re-ranker trained on 5- and 10-week histories works on the 20-week test history; only the Kaggle score speaks to this.

Session log reference.
The attached session export is the original, unmodified file from Claude Code.

References

[1] T. H. Haveliwala, “Topic-sensitive PageRank,” in Proc. 11th Int. World Wide Web Conf. (WWW), Honolulu, HI, USA, May 2002. [Online]. Available: https://archives.iw3c2.org/www2002/CDROM/refereed/127.html [Accessed: 2026-10-30]

[2] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in Recommender Systems Handbook, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. New York, NY, USA: Springer, 2011, pp. 73-105. [Online]. Available: https://doi.org/10.1007/978-0-387-85820-3_3 [Accessed: 2026-10-30]

[3] C. J. C. Burges, “From RankNet to LambdaRank to LambdaMART: An overview,” Microsoft Research, Redmond, WA, USA, Tech. Rep. MSR-TR-2010-82, 2010. [Online]. Available: https://www.microsoft.com/en-us/research/?p=159784 [Accessed: 2026-10-30]

[4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 3146-3154. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf [Accessed: 2026-10-30]

[5] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, Boston, MA, USA, Jul. 2009, pp. 758-759. [Online]. Available: https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf [Accessed: 2026-10-30]

[6] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 4765-4774. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf [Accessed: 2026-10-30]

[7] Kaggle, “sweet.tv - TV program recommender,” community prediction competition, 2020. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender/ [Accessed: 2026-10-30]